یک برنامه‌نویس می‌توانست با گماردن نواحی متفاوت آرایه‌ی v به هر کدام از هسته‌های پردازشی، موازی‌سازی داده‌ها (بخش ۴-۲-۲) را به‌صورت دستی روی یک سیستم چندهسته‌ای اِعمال کند؛ با این وجود، این کار، تکنیک دست‌یابی به موازی‌سازی را به‌صورت تنگاتنگ به سخت‌افزار فیزیکی گره می‌زند و لازم است که الگوریتم اصلاح شود و به‌تعداد هسته‌های پردازشی بر روی هر معماری خاصی دومرتبه کامپایل شود.

به‌روشی دیگر، یک برنامه‌نویس می‌توانست از TBB استفاده کند که الگوی parallel_for را فراهم می‌کند که دو مقدار انتظار دارد:

```
parallel_for (range body )
```

که *range* به بازه‌ی عناصری ارجاع می‌کند که تکرار خواهند شد (تحت عنوان **فضای تکرار**[1]) و *body* عملیاتی را مشخص می‌کند که بر روی زیربازه‌ای از عناصر انجام خواهد شد.

اکنون می‌توانیم سری بالا برای حلقه را با استفاده از الگوی parallel_for به‌صورت زیر بنویسیم:

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```

دو پارامتر اول مشخص می‌کنند که فضای تکرار از 0 تا n – 1 می‌باشد (که متناظر تعداد عناصر آرایه‌ی v می‌باشد). پارامتر دوم، یک تابع lambda به‌زبان C++ است که کمی توضیح لازم دارد. عبارت [=] (size_t i) پارامتر i می‌باشد که هر یک از مقادیر فضای تکرار را فرض دارد (در اینجا از 0 تا n - 1). هر مقدار i استفاده می‌شود تا مشخص کند که کدام عنصر v به‌عنوان یک پارامتر به تابع apply(v[i]) ارسال می‌شود.

کتابخانه‌ی TBB تکرارهای حلقه را به «قالب‌هایی» تقسیم خواهد کرد و تعدادی وظیفه ایجاد می‌کند که روی آن قالب‌ها عمل می‌کنند. (تابع parallel_for به برنامه‌نویسان اجازه می‌دهد تا به‌صورت دستی اندازه‌ی قالب‌ها را مشخص کنند در صورتی که تمایل داشته باشند.) TBB تعدادی نخ نیز ایجاد کرده و وظایف را به نخ‌های در دسترس خواهد گماشت. این موضوع کاملاً مشابه کتابخانه‌ی fork-join در جاوا می‌باشد. حسن این رویکرد در این است که تنها لازم دارد برنامه‌نویسان مشخص کنند چه عملیاتی می‌توانند اجرای موازی داشته باشند (با توصیف حلقه‌ی parallel_for) و کتابخانه است که جزئیات مشمول تقسیم کار به وظایف جداگانه را که اجرای موازی دارند مدیریت می‌کند. TBB اینتل داری هر دو نسخه‌ی تجاری و متن‌باز می‌باشد که بر روی ویندوز، لینوکس، و macOS اجرا می‌شوند.

۴-۶ ملاحظات نخ‌سازی

در این بخش، برخی ملاحظات مربوط به طراحی برنامه‌های چندنخی را بحث می‌کنیم.

۴-۶-۱ فراخوانی‌های سیستمی fork() و exec()

در فصل ۳، چگونگی استفاده از فراخوانی سیستمی ()fork را برای ایجاد کپی مجزایی از یک

1. Iteration space

پردازه بیان کردیم. معانی فراخوانی‌های سیستمی ()fork و ()exec در برنامه‌های چندنخی تغییر می‌کنند.

اگر یک نخ در برنامه‌ای ()fork را فراخوانی نماید، آیا پردازه‌ی جدید تمامی نخ‌ها را تکثیر می‌کند، یا پردازه‌ی جدید تک‌نخی خواهد بود؟ برخی سیستم‌های یونیکس برای انتخاب ()fork دو نسخه دارند، یکی تمامی نخ‌ها را تکثیر می‌کند و دیگری تنها نخی را که فراخوانی سیستمی ()fork را احضار کرده، تکثیر می‌کند.

فراخوانی سیستمی ()exec نوعاً به همان شیوه‌ای که در فصل ۳ بیان شد کار می‌کند. یعنی، اگر نخی فراخوانی سیستمی ()exec را احضار کند، برنامه‌ای که در پارامتر ارسالی به ()exec مشخص شده است جایگزین کل پردازه—اعم از تمامی نخ‌ها—خواهد شد.

این که از کدام یک از نسخه‌های ()fork استفاده شود به برنامه‌ی کاربردی آن بستگی دارد. اگر بلافاصله بعد از ()fork، ()exec فراخوانی شود، در این صورت تکثیر تمامی نخ‌ها ضرورتی ندارد، چرا که برنامه‌ی مشخص شده در پارامترهای ()exec، جایگزین کل پردازه خواهد شد. در اینجا، تنها تکثیر نخ فراخوانی‌کننده کفایت می‌کند. با این وجود، اگر پردازه‌ی جدا شده، بعد از تولد، ()exec را فراخوانی ننماید، پردازه‌ی جدا شده بایستی تمامی نخ‌ها را تکثیر کند.

۴-۶-۲ رسیدگی به سیگنال

از **سیگنال** در سیستم‌های یونیکس برای اطلاع یک پردازه از رخداد یک رویداد خاص استفاده می‌شود. هر سیگنال بسته به مبدأ آن و دلیل سیگنال رویداد می‌تواند به صورت همگام و یا ناهمگام تحویل شود. تمامی سیگنال‌ها فارغ از این که همگام یا ناهمگام باشند از الگوی یکسانی پیروی می‌کنند:

۱- هر سیگنال با وقوع رویداد خاصی تولید می‌شود.
۲- سیگنال تولید شده به یک پردازه تحویل می‌شود.
۳- همین که سیگنال تحویل شد، بایستی رسیدگی شود.

نمونه‌هایی از سیگنال‌های همگام عبارتند از: دسترسی غیرقانونی حافظه و تقسیم بر ۰. اگر برنامه‌ای که اجرا دارد، یکی از این اعمال را انجام بدهد، یک سیگنال تولید می‌شود. سیگنال‌های همگام، تحویل همان پردازه‌ای می‌شوند که عملیات آن باعث پیدایش سیگنال شده است (به همین خاطر است که همگام محسوب می‌شوند).

زمانی‌که سیگنالی توسط رویدادی در بیرون از یک پردازه‌ی در حال اجرا تولید می‌شود، پردازه سیگنال را به صورت ناهمگام دریافت می‌کند. نمونه‌هایی از این قبیل سیگنال‌ها عبارتند از: ختم یک پردازه با فشردن کلیدهای خاص (نظیر <control> <C>) و انقضای مهلت زمانی تایمر. نوعاً، سیگنال ناهمگام به پردازه‌ای دیگر فرستاده می‌شود.

هر سیگنال می‌تواند توسط یکی از دو راه‌انداز زیر *رسیدگی* شود:

۱- با راه‌انداز پیش‌فرض سیگنال
۲- با راه‌انداز سیگنال تعریفی کاربر

هر سیگنال یک **راه‌انداز سیگنال پیش‌فرض**[1] دارد که در زمان رسیدگی به سیگنال توسط هسته اجرا می‌شود. این عمل پیش‌فرض می‌تواند توسط یک **راه‌انداز سیگنال تعریفی کاربر**[2] که برای رسیدگی به سیگنال فراخوانی می‌شود از نو رانده شود. روش‌های متفاوتی در رسیدگی به سیگنال‌ها وجود دارند. برخی سیگنال‌ها ممکن است به‌سادگی چشم‌پوشی شوند، در حالی که، سیگنال‌های دیگری (همچون، یک دسترسی غیرقانونی به حافظه) هستند که با ختم برنامه رسیدگی می‌شوند.

رسیدگی به سیگنال‌ها در برنامه‌های تک‌نخی سر راست است: سیگنال‌ها همواره به یک پردازه تحویل می‌شوند. با این همه، تحویل سیگنال‌ها در برنامه‌های چندنخی بسیار پیچیده‌تر است که در آن پردازه می‌تواند نخ‌های متعددی داشته باشد. پس، سیگنال می‌بایست به کجا تحویل شود؟

در حالت کلی، انتخاب‌های زیر امکان‌پذیرند:

۱- سیگنال به نخی تحویل می‌شود که به آن می‌خورد.
۲- سیگنال به هر یک از نخ‌های پردازه تحویل می‌شود.
۳- سیگنال به نخ‌های خاصی از پردازه تحویل می‌شود.
۴- نخ مشخصی برای دریافت تمامی سیگنال‌های پردازه گمارده می‌شود.

روش تحویل سیگنال به نوع سیگنال تولید شده بستگی دارد. برای مثال، سیگنال‌های همگام لازم است، تحویل نخی شوند که باعث سیگنال بوده است و نباید به نخ‌های دیگر پردازه تحویل گردند. اما، این وضعیت در مورد سیگنال‌های ناهمگام، چندان روشن نیست. برخی سیگنال‌های ناهمگام (مانند سیگنالی که پردازه‌ای را خاتمه می‌دهد، برای مثال <control> <C>) بایستی به تمامی نخ‌ها فرستاده شوند.

تابع استاندارد یونیکس برای تحویل یک سیگنال به صورت زیر است:

```
kill(pid_t pid, int signal)
```

این تابع پردازه‌ای (pid) را که قرار است یک سیگنال خاص (signal) به آن تحویل شود مشخص می‌کند. اکثر نسخه‌های چندنخی یونیکس به یک نخ امکان می‌دهند که سیگنال‌هایی را که خواهد پذیرفت و آنهایی را که مسدود خواهد کرد، مشخص کند. بنابراین، در برخی موارد، یک سیگنال ناهمگام ممکن است تنها به آن دسته از نخ‌هایی که آن را مسدود نمی‌کنند، تحویل شود. با این وجود، چون لازم است که سیگنال‌ها فقط یک بار رسیدگی شوند، سیگنال نوعاً تنها به اولین نخ پیدا شده که آن را مسدود نمی‌کند، تحویل می‌شود. نخ‌های پوزیکس، تابع زیر را فراهم می‌کنند که به یک سیگنال اجازه می‌دهد تا به یک نخ مشخص (tid) تحویل شود:

```
pthread_kill(pthread_t tid, int signal)
```

1. Default signal handler
2. User-defined signal handler

هرچند که ویندوز آشکارا از سیگنال‌ها پشتیبانی نمی‌کند، این امکان را به ما می‌دهد که آنها را با استفاده از **فراخوانی ناهمگام روال‌ها**[1] (**APC**) تقلید کنیم. امکان APC نخ کاربر را قادر می‌سازد تابعی را مشخص کند که در زمان دریافت هشداری توسط نخ کاربر مبنی بر یک رویداد خاص فراخوانی شود. همان‌گونه که از نام آن بر می‌آید، APC تقریباً معادل سیگنال ناهمگام یونیکس است. با این وجود، در حالی‌که یونیکس با چگونگی راه‌اندازی سیگنال‌ها در یک محیط چندنخی درگیر است، امکان APC هموارتر است، چرا که APC به جای یک پردازه به نخ خاصی تحویل می‌گردد.

۳-۶-۴ برکناری نخ

برکناری نخ[2] شامل ختم یک نخ، قبل از تکمیل آن است. برای مثال، اگر نخ‌های متعددی یک بانک‌اطلاعاتی را همروند جستجو می‌کنند و یکی از آنها نتیجه را برگرداند، می‌توان باقی نخ‌ها را کنار گذاشت. وضعیت مشابهی در زمانی روی می‌دهد که کاربر دکمه‌ای از مرورگر وب را فشار می‌دهد تا صفحه‌ی وب را متوقف نموده و نگذارد که بارگذاری بیشتری انجام شود. صفحه‌ی وب اغلب با استفاده از چندین نخ بارگذاری می‌شود—هر تصویر با نخ مجزایی بار می‌شود. زمانی که کاربر دکمه‌ی stop را فشار می‌دهد، تمامی نخ‌های بارکننده‌ی صفحه لغو می‌شوند.

نخی که قرار است برکنار شود اغلب به نام **نخ هدف**[3] از آن یاد می‌شود. برکناری نخ هدف می‌تواند در دو سناریوی متفاوت اتفاق بیفتد که عبارتند از:

۱- **برکناری ناهمگام**[4]: یک نخ بلافاصله نخ هدف را خاتمه می‌دهد.

۲- **برکناری تأخیردار**[5]: نخ هدف با تناوبی بررسی می‌کند تا ببیند که آیا زمان خاتمه‌اش رسیده است، با این کار، این شانس را دارد که به شیوه‌ای منظم خود را خاتمه بدهد.

مشکلی که گاهی در برکناری نخ‌ها پیش می‌آید شرایطی است که نخ برکنارشونده منابعی در اختیار دارد یا در میانه‌ی به روز کردن داده‌هایی است که با سایر نخ‌ها به طور مشترک استفاده می‌شوند. این مشکل مخصوصاً خود را در زمان برکناری ناهمگام نشان می‌دهد. سیستم‌عامل در اغلب موارد، منابع سیستمی را از نخ برکنارشده باز پس می‌گیرد، اما تمامی منابع را پس نمی‌گیرد. بنابراین، با برکناری یک نخ به صورت ناهمگام ممکن است که یک منبع موردنیاز سطح سیستم باز هم آزاد نشود.

در مقایسه، در برکناری تأخیردار، یک نخ خبر می‌دهد که قرار است نخ هدفی برکنار شود، اما برکناری به تأخیر می‌افتد تا این که نخ هدف، با بررسی پرچم برکناری خود متوجه می‌شود که آیا می‌تواند برکنار شود یا خیر. این کار به نخ اجازه می‌دهد تا بررسی کند که آیا نقطه‌ای که در آن برکنار می‌شود مناسب خواهد بود یا خیر.

---

1. Asynchronous procedure calls
2. Thread cancellation
3. Target thread
4. Asynchronous cancellation
5. Deferred cancellation

برکناری نخ در نخ‌های پوزیکس با استفاده از تابع pthread_cancel() آغاز می‌شود. شناسه‌ی نخ هدف به عنوان پارامتری به این تابع ارسال می‌شود. کد زیر، ایجاد–و سپس برکناری–یک نخ را نشان می‌دهد:

```
pthread_t tid;
/* create the thread */
pthread_create(&tid, 0, worker, NULL);

        . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```

با این وجود، احضار pthread_cancel() فقط درخواست برکناری نخ هدف را نشان می‌دهد؛ برکناری واقعی به این بستگی دارد که نخ هدف چگونه برپا شده تا به این درخواست رسیدگی کند. موقعی که نهایتاً نخ هدف برکنار می‌شود، فراخوانی به pthread_join() در نخ برکارکننده بر می‌گردد. نخ‌های پوزیکس از سه مد برکناری پشتیبانی می‌کنند. هر مد به صورت یک حالت و یک نوع تعریف می‌شود، چنان که در جدول زیر نشان داده می‌شود. یک نخ می‌تواند حالت و نوع برکناری خود را با استفاده از یک API انتخاب کند.

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

همان‌گونه که جدول نشان می‌دهد، نخ‌های پوزیکس به نخ‌ها اجازه می‌دهند تا برکناری را غیرفعال یا فعال سازند. پر واضح است که یک نخ در صورتی که برکناریش غیرفعال باشد، نمی‌تواند برکنار شود. با این وجود، درخواست‌های برکناری معوق می‌مانند، لذا نخ می‌تواند بعداً برکناری را فعال کرده و به درخواست پاسخ دهد.

نوع پیش‌فرض برکناری، برکناری معوقه است. در اینجا، برکناری فقط زمانی اتفاق می‌افتد که یک نخ به یک **نقطه‌ی برکناری** برسد. اکثر فراخوانی‌های سیستمی مسدودکننده در پوزیکس و کتابخانه‌ی استاندارد C به‌صورت نقاط برکناری تعریف می‌شوند، و موقعی که در یک سیستم لینوکس، فرمان man pthreads احضار می‌شود این موارد فهرست می‌شوند. برای مثال، فراخوانی سیستمی read() یک نقطه‌ی برکناری است که به یک نخی که مسدود است اجازه می‌دهد در حالی که به‌انتظار ورودی read() می‌باشد برکنار شود.

یک تکنیک برای برپایی یک نقطه‌ی برکناری، احضار تابع pthread_testcancel() است. در صورتی که یک درخواست برکناری معوقه پیدا شود، فراخوانی به pthread_testcalcel() بر نخواهد

گشت و نخ خاتمه می‌یابد؛ در غیر این صورت، فراخوانی به تابع باز خواهد گشت، و نخ به اجرایش ادامه خواهد داد. به‌علاوه، اگر نخی برکنار شود، نخ‌های پوزیکس اجازه می‌دهند تا یک تابعی تحت عنوان **cleanup handler** احضار شود. این تابع اجازه می‌دهد هرگونه منبعی که یک نخ کسب کرده باشد، قبل از این که نخ خاتمه یابد، رها گردد.

کد زیر نشان می‌دهد که چگونه یک نخ ممکن است به یک درخواست برکناری با استفاده از برکناری معوقه پاسخ دهد:

```
while (1) {
        /* do some work for awhile */
        /* . . . */

        /* check if there is a cancellation request */
        pthread_testcancel();
}
```

به خاطر ملاحظاتی که پیشتر عنوان شد، برکناری ناهمگام در مستندات نخ‌های پوزیکس توصیه نمی‌شود. بنابراین، در اینجا آن را پوشش نمی‌دهیم. یک نکته‌ی جالب این است که در سیستم‌های لینوکس، برکناری نخ با استفاده از API نخ‌های پوزیکس از طریق سیگنال‌ها (بخش ۴-۶-۲) راه‌اندازی می‌شود.

برکناری نخ در جاوا از یک سیاستی شبیه به برکناری تأخیردار نخ‌های پوزیکس استفاده می‌کند. برای برکناری یک نخ در جاوا بایستی متد interrupt() را احضار کنید که وضعیت توقف نخ هدف را true قرار می‌دهد:

```
Thread worker;

. . .

/* set the interruption status of the thread */
worker.interrupt()
```

یک نخ می‌تواند وضعیت توقف خود را با احضار متد isInterrupted() چک کند که یک مقدار بولی از وضعیت توقف نخ بر می‌گرداند:

```
while (!Thread.currentThread().isInterrupted()) {
        . . .
}
```

۴-۶-۴ حافظه‌ی محلی نخ

نخ‌های متعلق به یک پردازه، داده‌های پردازه را به اشتراک می‌گیرند. در حقیقت، این اشتراک داده‌ها یکی از منافع برنامه‌نویسی چندنخی را فراهم می‌کند. با این وجود، در بعضی موارد، هر یک از نخ‌ها کپی داده‌های خاص خود را لازم دارند. این قبیل داده‌ها را **حافظه‌ی محلی نخ** (**TLS**)[1]

1. Thread-local storage

می‌نامیم. برای مثال، در یک سیستم پردازش تراکنش می‌توانیم هر یک از تراکنش‌ها را در یک نخ مجزا سرویس بدهیم. به علاوه، به هر یک از تراکنش‌ها می‌توان شناسه‌ی واحدی نسبت داد. برای مرتبط‌ساختن هر نخ به شناسه‌ی تراکنش واحد خود می‌توانیم از حافظه‌ی محلی نخ استفاده کنیم.

به سادگی ممکن است TLS با متغیرهای محلی موجب سردرگمی شود. با این وجود، متغیرهای محلی فقط در طی احضار یک تابع منفرد قابل رؤیت هستند، در حالی‌که داده‌های TLS مابین احضار توابع قابل رؤیت هستند. به‌علاوه، موقعی که برنامه‌نویس هیچ کنترلی نسبت به فرایند ایجاد نخ ندارد—برای مثال، موقع استفاده از یک تکنیک ضمنی همچون انبار نخ—در این صورت، یک رویکرد جایگزین ضرورت دارد.

در برخی جهات، TLS شبیه به داده‌های static است؛ تفاوت در این است که داده‌های TLS برای هر یک از پردازه‌ها یکتا هستند. (در حقیقت، TLS معمولاً به‌صورت static اعلان می‌شود.) اکثر کتابخانه‌های نخ و کامپایلرها، TLS را پشتیبانی می‌کنند. برای مثال، جاوا کلاس ThreadLocal<T> را با متدهای set() و get() برای اشیاء ThreadLocal<T> فراهم می‌کند. نخ‌های پوزیکس شامل نوع pthread_key_t می‌باشند که یک کلید که خاص هر نخ است فراهم می‌کند. در ادامه، این کلید می‌تواند برای دسترسی به داده‌های TLS استفاده شود. زبان C# مایکروسافت به‌سادگی لازم دارد که صفت حافظه‌ی [ThreadStatic] افزوده شود تا داده‌های محلی نخ را اعلان کند. کامپایلر gcc کلیدواژه‌ی کلاس حافظه‌ی thread_ را برای اعلان داده‌های TLS فراهم می‌کند. برای مثال، اگر ما تمایل داشته باشیم که یک شناسه‌ی یکتا برای هر نخ انتساب کنیم، آن را به صورت زیر اعلان خواهیم کرد:

```
static __thread int threadID;
```

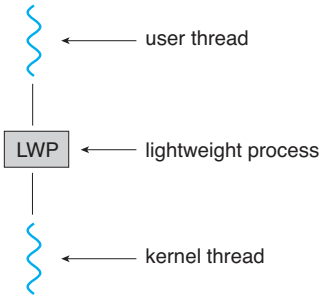
۴-۶-۵ فعال‌سازی زمانبندها

آخرین ملاحظه‌ای که در رابطه با برنامه‌های چندنخی مطرح می‌شود به ارتباط میان هسته و کتابخانه‌ی نخ مربوط است که ممکن است توسط مدل‌های چند-به-چند و دوسطحی که در بخش ۴-۳-۳ بحث شدند، لازم شود. این قبیل هماهنگی امکان می‌دهد تعداد نخ‌های هسته به صورت پویا تنظیم شوند که کمکی هم برای تضمین بهترین کارایی خواهد بود.

بسیاری از سیستم‌ها که مدل چند-به-چند و یا دوسطحی را پیاده‌سازی می‌کنند یک ساختمان‌داده‌ی میانی بین نخ‌های کاربر و هسته قرار می‌دهند. این ساختمان‌داده—که نوعاً به **پردازه‌ی سبک وزن**[1] یا **LWP** معروف است—در شکل ۴-۲۰ نشان داده می‌شود. برای کتابخانه‌ی نخ کاربر، LWP به یک پردازنده‌ی مجازی می‌ماند که برنامه‌ی کاربردی می‌تواند یک نخ کاربر را برای اجرا بر روی آن زمانبندی کند. هر LWP به یک نخ هسته متصل می‌شود و این نخ‌های هسته هستند که سیستم‌عامل جهت اجرا در پردازنده‌های فیزیکی زمانبندی می‌نماید. اگر نخ هسته

1. Lightweight process

مسدود شود (برای مثال، زمانی‌که منتظر تکمیل عملیات I/O می‌باشد)، LWP نیز مسدود می‌گردد. در بالای این زنجیره، نخ سطح کاربر متصل به LWP نیز مسدود می‌گردد.



شکل ۴-۲۰ پردازه‌ی سبک وزن (LWP).

یک برنامه‌ی کاربردی برای اجرای کارآمد امکان دارد که به هر تعداد LWP نیاز پیدا کند. یک برنامه‌ی کاربردی پابند پردازنده را با اجرا بر روی یک تک‌پردازنده در نظر بگیرید. در این شرایط، تنها یک نخ می‌تواند در هر لحظه اجرا داشته باشد، لذا یک LWP کافی است. با این وجود، یک برنامه‌ی کاربردی که تمایل بیشتری به I/O دارد، برای اجرا به چندین LWP نیاز خواهد داشت. نوعاً، برای هر فراخوانی سیستمی همروند مسدودشونده یک LWP لازم است. برای مثال، فرض کنید همزمان پنج درخواست متفاوت خواندن فایل می‌رسند. پنج LWP مورد نیاز است، چون تمامی آنها می‌توانند به انتظار تکمیل I/O بمانند. اگر پردازه‌ای تنها چهار LWP داشته باشد، در این صورت بایستی پنجمین درخواست منتظر شود تا یکی از LWPها از هسته برگردد.

یک طرح برای ارتباط میان کتابخانه‌ی نخ کاربر و هسته به **فعال‌سازی زمانبند**[1] معروف است. این طرح به این صورت کار می‌کند: هسته برای برنامه‌ی کاربردی، مجموعه‌ای از پردازنده‌های مجازی (LWP) را فراهم می‌کند و برنامه‌ی کاربردی می‌تواند نخ‌های کاربر را به روی یک پردازنده‌ی مجازی در دسترس زمانبندی کند. به علاوه، هسته بایستی برنامه‌ی کاربردی را از وقوع رویدادهای خاصی مطلع سازد. این روال به **ندا**[2] معروف است. نداها توسط کتابخانه‌ی نخ با یک **راه‌انداز ندا**[3] رسیدگی می‌شوند و راه‌اندازهای ندا بایستی بر روی یک پردازنده‌ی مجازی اجرا شوند. یک رویداد که ندا به راه می‌اندازد زمانی رخ می‌دهد که یک نخ برنامه‌ی کاربردی در آستانه‌ی انسداد است. در اینجا، هسته برای برنامه‌ی کاربردی یک ندا می‌فرستد تا برنامه‌ی کاربردی را مطلع سازد که نخی در حال انسداد است، ضمن این که آن نخ را هم مشخص می‌کند. سپس، هسته یک پردازنده‌ی مجازی جدید به برنامه‌ی کاربردی تخصیص می‌دهد. برنامه‌ی کاربردی یک راه‌انداز ندا بر روی این پردازنده‌ی مجازی جدید اجرا می‌کند که، حالت نخ مسدود را ذخیره کرده و پردازنده‌ی مجازی را که نخ مسدود در آن اجرا داشت رها می‌سازد. راه‌انداز ندا، نخ دیگری را که برای اجرا در پردازنده‌ی مجازی جدید آمادگی دارد زمانبندی می‌کند. زمانی‌که

1. Scheduler activation
2. Upcall
3. Upcall handler

رویداد نخ مسدودی که منتظر آن بود اتفاق می‌افتد، هسته ندای دیگری به کتابخانه‌ی نخ می‌فرستد تا اطلاع بدهد که هم‌اکنون نخ مسدود پیشین می‌تواند اجرا شود. راه‌انداز ندای این رویداد نیز به یک پردازنده‌ی مجازی نیاز خواهد داشت که هسته می‌تواند پردازنده‌ی مجازی جدیدی تخصیص بدهد یا یکی از نخ‌های کاربر را پس‌گرفته و راه‌انداز ندا را بر روی پردازنده‌ی مجازی آن اجرا کند. بعد از این که، نخ از حالت انسداد خارج شد و برای آمادگی اجرا علامت خورد، برنامه‌ی کاربردی، نخ آماده را به منظور اجرا بر روی یکی از پردازنده‌های مجازی در دسترس زمانبندی می‌نماید.

۴-۷ مثال‌های سیستم‌عاملی

تا اینجا، یک سری مفهوم و ملاحظات مربوط به نخ‌ها را بررسی کردیم. این فصل را با مرور چگونگی پیاده‌سازی نخ‌ها در سیستم‌های ویندوز و لینوکس جمع می‌بندیم.

۴-۷-۱ نخ‌های ویندوز

یک برنامه‌ی کاربردی ویندوز به صورت پردازه‌ای مجزا اجرا می‌شود و هر پردازه می‌تواند یک یا چندین نخ را شامل شود. API ویندوز برای ایجاد نخ‌ها در بخش ۴-۴-۲ پوشش داده شد. به علاوه، ویندوز از نگاشت یک-به-یک مطرح شده در بخش ۴-۳-۲ استفاده می‌کند که هر نخ سطح کاربر به نخی از هسته نگاشته می‌شود.

مؤلفه‌های کلی یک نخ عبارتند از:

- یک ID نخ که نخ را به صورت یکتا مشخص می‌کند.
- یک مجموعه ثبات که وضعیت پردازنده را نشان می‌دهد.
- شمارنده‌ی برنامه
- پشته‌ی کاربر، که موقع اجرای نخ در مد کاربر به‌کار می‌رود، و پشته‌ی هسته، که موقع اجرای نخ در مد هسته کاربرد دارد.
- یک ناحیه‌ی حافظه‌ی خصوصی که توسط کتابخانه‌های مختلف زمان اجرا و کتابخانه‌های اتصالی پویا[1] (DLL) استفاده می‌شود.

مجموعه ثبات، پشته‌ها و ناحیه‌ی خصوصی حافظه به **متن**[2] نخ معروفند.

ساختمان‌داده‌های اصلی یک نخ عبارتند از:

- ETHREAD□بلاک اجرایی نخ
- KTHREAD□بلاک نخ هسته
- TEB□بلاک محیط نخ

مؤلفه‌های کلیدی ETHREAD شامل اشاره‌گری به پردازه‌ی تعلق نخ و آدرس روالی است که نخ در آن کنترل را به‌دست می‌گیرد. همچنین ETHREAD، یک اشاره‌گر به KTHREAD متناظر دارد.

1. Dynamic link libraries
2. Context